CHAPTER 11

Cryptography 101

In this chapter, you will

- Describe cryptography and encryption techniques
- Define cryptographic algorithms
- Describe public and private key generation concepts
- Describe digital signature components and usage
- Describe cryptanalysis and code-breaking tools and methodologies
- List cryptography attacks

Around 180 BC, the Greek philosopher and historian Polybius was busy putting together some revolutionary re-thinking of government. He postulated on such ideas as the separation of powers and a government meant to serve the people instead of rule over them. If this sounds familiar, it should: his work became part of the foundation for later philosophers and writers (including Montesquieu), not to mention the U.S. Constitution.

Considering, though, the times he lived in, not to mention his family circumstances and upbringing, it's fairly easy to see where Polybius might have wanted a little secrecy in his writing. His father was a Greek politician and an open opponent of Roman control of Macedonia. This eventually led to his arrest and imprisonment, and Polybius was deported to Rome. There, Polybius was employed as a tutor. He eventually met and befriended a Roman military leader and began chronicling the events he witnessed (these works would become known as *The Histories,* detailing the Roman rise to power from 264 to 146 BC).

During all this historical writing, though, he couldn't shake his father's voice and continued writing about the separation of government powers and the abuses of dictatorial rule. In an effort to keep this part of his writing secret, he came up with what has become known as the *Polybius square*. The idea was simple. First, create a checkerboard with numbers running across the top and along the left side. Next, populate the interior with the letters of the alphabet. Then, when writing, a letter would become its coordinates on the grid; for example, *A* might be written as 11, while *B* would be 12.

	1	2	3	4	5	6
1	A	B	C	D	E	F
2	G	H	I	J	K	L
3	M	N	O	P	Q	R
4	S	T	U	V	W	X
5	Y	Z	0	1	2	3
6	4	5	6	7	8	9

Was it an unbeatable cypher system that kept everything safe? Was it even the first recorded effort at encrypting messages so that no one but the recipient could read them? No, it was neither. It did, however, mark one of the historic turning points in cryptography and led to worlds of other inventions and uses (including steganography). From cavemen working out a succession of knocks and beats to the secure e-mail I just sent my boss a few minutes ago, we've been trying to keep things secret since the dawn of time. And, since the dawn of time, we've been trying to figure out what the other guy was saying—trying to "crack his code." The implementation and study of this particular little fascination of the human psyche—securing communication between two or more parties—is known as *cryptography*. For you budding ethical hackers reading this book, the skill you're looking to master,

though, is *cryptanalysis,* which is the study and methods used to crack encrypted communications.

CRYPTOGRAPHY AND ENCRYPTION OVERVIEW

I debated long and hard over just how much history to put into this discussion on cryptography but finally came to the conclusion I shouldn't put in any, even though it's *really* cool and interesting (c'mon, admit it, the opening to this chapter entertained and enthralled you, didn't it?). I mean, you're probably not concerned with how the ancient Romans tried to secure their communications or who the first purveyors of *steganography*—hiding messages inside an image—were (toss-up between the Greeks and the Egyptians, depending on your source). What you are, and should be, concerned with is what cryptography actually is and why you should know anything about it. Excellent thoughts. Let's discuss.

Cryptography is the science or study of protecting information, whether in transit or at rest, by using techniques to render the information unusable to anyone who does not possess the means to decrypt it. The overall process is fairly simple: take *plain-text* data (something you can read), apply a cryptographic method, and turn it into *cipher text* (something you can't read)—so long as there is some provision to allow you to bring the cipher text back to plain text. What is not so simple is the actual process of encrypting and decrypting.

Cryptanalysis is the study and methods used to crack the communications we just talked about, and there are three main methods of effort to discuss. First, you can attack encrypted communication in a linear fashion—that is, take blocks of known text and compare them to blocks of the encrypted text, line by line, from front to back. *Linear cryptanalysis* works best on block ciphers (something we're going to cover a little later on) and was developed by Mitsuru Matsui in 1993. As encryption techniques evolved, so did cryptanalysis, and *differential*

cryptanalysis came about. This one is applicable to symmetric key algorithms and basically compares differences in inputs to how each one affects the outcome. *Integral cryptanalysis,* the last of our three methods, borrows from differential in that the input versus output comparison remains; however, integral runs multiple computations of the same block size input.

Now, will knowing the approach used for a specific cryptanalysis effort make you a better pen tester? Who knows—maybe some of you reading this will go into that line of work and this will be insanely helpful as a starter. For the rest of us, it's good info to know, but not really a life-changing experience. So sit back and relax as the rest of this chapter is dedicated to exploring some of the mathematical procedures, known as *encryption algorithms* or *ciphers,* used to encrypt and decrypt data.

NOTE Don't be confused by the term *plain text*. Yes, it can be used to define text data in ASCII format. However, within the confines of cryptography, plain text refers to anything that is not encrypted—whether text or not.

It's also important to understand what functions cryptography can provide. In Chapter 1, we discussed the hallowed trinity of security—confidentiality, integrity, and availability. When it comes to cryptography, confidentiality is the one that most often is brought up. Encrypting data helps to provide confidentiality of the data because only those with the "key" can see it. However, some other encryption algorithms and techniques also provide for integrity (hashes that ensure the message hasn't been changed) as well as a new term we have yet to discuss here: *nonrepudiation,* which is the means by which a recipient can ensure the identity of the sender and neither party can deny having sent or

received the message. Our discussion of public key infrastructure (PKI) later will definitely touch on this. This chapter is all about defining what cryptography methods are available so that you know what you're up against as an ethical hacker.

Encryption Algorithms and Techniques

Cryptographic systems can be as simple as substituting one character for another (the old Caesar Cipher simply replaced characters in a string: *B* for *A*, *C* for *B*, and so on) or as complex as applying mathematical formulas to change the content entirely. Modern-day systems use encryption algorithms and separate keys to accomplish the task. In its simplest definition, an *algorithm* is a step-by-step method of solving a problem. The problem, when it comes to the application of cryptography, is how do you render something unreadable and then provide a means to recover it? Encryption algorithms were created for just such a purpose.

NOTE Encryption of bits takes, generally, one of two different forms: substitution or transposition. Substitution is exactly what it sounds like—bits are simply replaced by other bits. Transposition doesn't replace bits at all; it changes their order altogether.

Encryption algorithms—mathematical formulas used to encrypt and decrypt data—are highly specialized and, sometimes, very complex. These algorithms are also known as ciphers. The good news for you as a CEH candidate is you don't need to learn the minutiae of how these algorithms actually accomplish their task. You will need to learn, however, how they are classified and some basic information about each one. For example, a good place to start might be the understanding that modern-day systems use encryption algorithms that are

dependent on a separate key, meaning that without the key, the algorithm itself should be useless in trying to decode the data. There are two main methods by which these keys can be used and shared: symmetric and asymmetric. Before we get to that, though, let's discuss how ciphers work.

All encryption algorithms on the planet have basically two methods they can use to encrypt data, and if you think about how they work, the names make perfect sense. In the first method, bits of data are encrypted as a continuous stream. In other words, readable bits in their regular pattern are fed into the cipher and are encrypted one at a time, usually by an XOR operation (exclusive-or). Known as *stream ciphers,* these work at a very high rate of speed.

In the other method, data bits are split up into blocks and fed into the cipher. Each block of data (commonly 64 bits at a time) is then encrypted with the key and algorithm. These ciphers, known as *block ciphers,* use methods such as substitution and transposition in their algorithms and are considered simpler, and slower, than stream ciphers.

NOTE Want to learn a little more about all this cryptography stuff? Why not give CrypTool (https://www.cryptool.org/en/) a shot? It's free, it's online, and it has multiple offshoots to satisfy almost all your cryptographic curiosity.

In addition to the types of ciphers, another topic you need to commit to memory applies to the nuts and bolts. XOR operations are at the core of a lot of computing. An XOR operation requires two inputs. In the case of encryption algorithms, this would be the data bits and the key bits. Each bit is fed into the operation—one from the data, the next from the key—and then XOR makes a

determination. If the bits match, the output is a 0; if they don't, it's a 1 (see the following XOR table).

First Input	Second Input	Output
0	0	0
0	1	1
1	0	1
1	1	0

For example, suppose you had a stream of data bits that read 10110011 and a key that started 11011010. If you did an XOR on these bits, you'd get 01101001. The first two bits (1 from data and 1 from the key) are the same, so the output is a zero (0). The second two bits (0 from data and 1 from the key) are different, outputting a one (1). Continue that process through, and you'll see the result.

In regard to cryptography and pure XOR ciphers, keep in mind that key length is of utmost importance. If the key chosen is actually smaller than the data, the cipher will be vulnerable to frequency attacks. In other words, because the key will be used repeatedly in the process, its very frequency makes guessing it (or using some other cryptanalytic technique) easier.

EXAM TIP There is a lot to remember in this chapter, so this tip may be helpful as a quick memorization effort. The way modern ciphers work can be discussed in two main categories: ciphers based on the type of key used and ones based on the type of input data. Key types include private (one key does it all) and public (different keys for encrypting and decrypting), while the type of input data refers to block (fixed-sized blocks encrypted) versus stream (continuous feed of data is encrypted as it arrives).

Symmetric Encryption

Also known as *single key* or *shared key, symmetric encryption* simply means one key is used both to encrypt and to decrypt the data. So long as both the sender and the receiver know/have the secret key, communication can be encrypted between the two. In keeping with the old acronym K.I.S.S. (Keep It Simple, Stupid), the simplicity of symmetric encryption is its greatest asset. As you can imagine, this makes things easy and fast. Bulk encryption needs? Symmetric algorithms and techniques are your best bet.

But symmetric key encryption isn't all roses and chocolate; there are some significant drawbacks and weaknesses. For starters, key distribution and management in this type of system are difficult. How do you safely share the secret key? If you send it over the network, someone can steal it. Additionally, because everyone has to have a specific key from each partner they want to communicate with, the sheer number of keys needed presents a problem.

Suppose you had two people you wanted to safely communicate with. This creates three different lines of communication that must be secured; therefore, you'd need three keys. If you add another person to the mix, there are now six lines of communication, requiring six different keys; see Figure 11-1. As you can imagine, this number jumps up exponentially the larger your network becomes. The formula for calculating how many key pairs you will need is

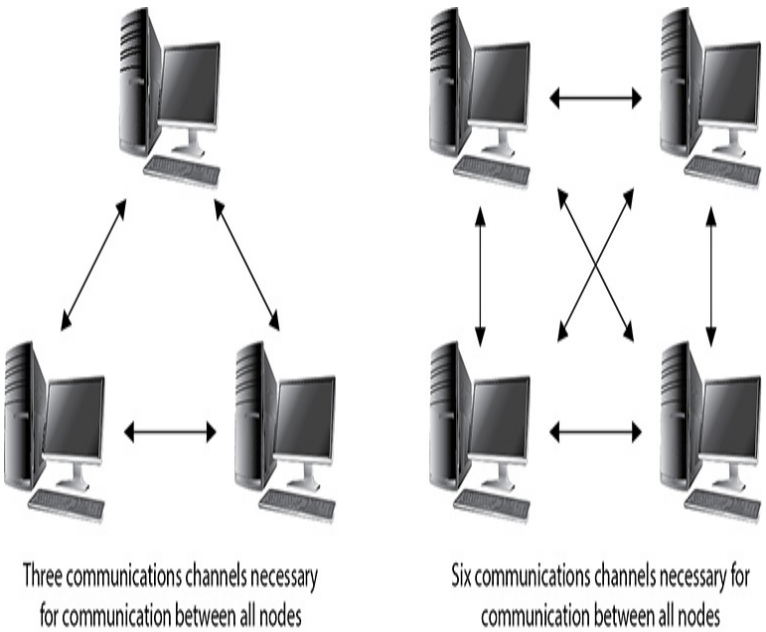


Figure 11-1 Key distribution in symmetric encryption systems

$$N\,(N - 1)\,/\,2$$

where N is the number of nodes in the network.

Here are some examples of symmetric algorithms:

- **DES** A block cipher that uses a 56-bit key (with 8 bits reserved for parity). Because of the small key size, this encryption standard became quickly outdated and is not considered a very secure encryption algorithm.
- **3DES** A block cipher that uses a 168-bit key. 3DES (called *triple* DES) can use up to three keys in a multiple-encryption method. It's much more effective than DES but is much slower.
- **AES (Advanced Encryption Standard)** A block cipher that uses a key length of 128, 192, or 256 bits, and effectively replaces DES. It's much faster than DES or 3DES.
- **IDEA (International Data Encryption Algorithm)** A block cipher that uses a 128-bit key and was also designed to replace DES.

Originally used in Pretty Good Privacy (PGP) 2.0, IDEA was patented and used mainly in Europe.

- **Twofish** A block cipher that uses a key size up to 256 bits.
- **Blowfish** A fast block cipher, largely replaced by AES, using a 64-bit block size and a key from 32 to 448 bits. Blowfish is considered public domain.
- **RC (Rivest Cipher)** Encompasses several versions, from RC2 through RC6. A block cipher that uses a variable key length up to 2040 bits. RC6, the latest version, uses 128-bit blocks and 4-bit working registers, whereas RC5 uses variable block sizes (32, 64, or 128) and 2-bit working registers.

And there you have it—symmetric encryption is considered fast and strong but poses some significant weaknesses. It's a great choice for bulk encryption because of its speed, but key distribution is an issue because the delivery of the key for the secured channel must be done offline. Additionally, scalability is a concern because the larger the network gets, the number of keys that must be generated increases greatly.

Lastly, symmetric encryption does a great job with confidentiality but does nothing to provide for another important security measure—nonrepudiation. As stated earlier, nonrepudiation is the method by which we can prove the sender's identity, as well as prevent either party from denying they took part in the data exchange. These weaknesses led to the creation and implementation of the second means of encryption—asymmetric.

Asymmetric Encryption

Asymmetric encryption came about mainly because of the problem inherent in using a single key to encrypt and decrypt messages—just how do you share the key efficiently and easily without compromising the security? The answer was, of course, to simply use two keys. In this key-pair system, both are generated together, with one

key used to encrypt a message and the other to decrypt it. The encryption key, also known as the *public key,* could be sent anywhere, to anyone. The decryption key, known as the *private key,* is kept secured on the system.

For example, suppose two people want to secure communications across the Internet between themselves. Using symmetric encryption, they'd need to develop some offline method to exchange the single key used for all encryption/decryption (and agree on changing it fairly often). With asymmetric encryption, they both generate a key pair. User A sends his public key to User B, and User B sends his public key to User A. Neither is concerned if anyone on the Internet steals this key because it can be used only to encrypt messages, not to decrypt them. This way, data can be encrypted by a key and sent without concern because the only method to decrypt it is the use of the private key belonging to that pair.

EXAM TIP Asymmetric encryption comes down to this: what one key encrypts, the other key decrypts. It's important to remember the public key is the one used for encryption, whereas the private key is used for decryption. Either can be used for encryption or decryption within the pair (as you'll see later in this chapter), but in general remember public = encrypt, private = decrypt.

In addition to addressing the concerns over key distribution and management, as well as scalability, asymmetric encryption addresses the nonrepudiation problem. For example, consider the following scenario: There are three people on a network—Bob, Susan, and Badguy—using asymmetric encryption. Susan wants to send an encrypted message to Bob and asks for a copy of his public key. Bob sees this request, and so does Badguy. Both send her a public key that says "Bob's Public Key." Susan is now confused because she does not know which key is the real one. So, how can they prove

to each other exactly who they are? How can Bob send a public key to Susan and have her, with some semblance of certainty, know it's actually from him?

NOTE It's important to note that although signing a message with the private key is the act required for providing a digital signature and, in effect, confidentiality and nonrepudiation, this is valid only if the keys are good in the first place. This is where key management and the certificate authority process comes into play—without their control over the entire scenario, none of this is worthwhile.

The answer, of course, is for Bob to send a message from his system encrypted with his private key. Susan can then attempt to decrypt the message using both public keys. The one that works must be Bob's actual public key because it's the only key in the world that could open a message encrypted with his private key. Susan, now happy with the knowledge she has the correct key, merrily encrypts the message and sends it on. Bob receives it, decrypts it with his private key, and reads the message. Meanwhile, Badguy weeps in a corner, cursing the cleverness of the asymmetric system. This scenario, along with a couple of other interesting nuggets and participants, illustrates the public key infrastructure framework we'll be discussing later in this chapter.

NOTE Simple public key infrastructure (PKI) systems are easy enough to understand, but if you've ever signed an e-mail with a key that doesn't match your actual sending address, things can get crazy. Assuming your PKI is a little more elegant, you can associate disparate keys (with different addresses) to an individual. However, things can get really out of hand really quickly. Can you *really* trust that signature?

Here are some examples of asymmetric algorithms:

- **Diffie-Hellman** Developed for use as a key exchange protocol, Diffie-Hellman is used in Secure Sockets Layer (SSL) and IPSec encryption. It can be vulnerable to man-in-the-middle attacks, however, if the use of digital signatures is waived.
- **Elliptic Curve Cryptosystem (ECC)** This uses points on an elliptical curve, in conjunction with logarithmic problems, for encryption and signatures. It uses less processing power than other methods, making it a good choice for mobile devices.
- **El Gamal** Not based on prime number factoring, this method uses the solving of discrete logarithm problems for encryption and digital signatures.
- **RSA** This is an algorithm that achieves strong encryption through the use of two large prime numbers. Factoring these numbers creates key sizes up to 4096 bits. RSA can be used for encryption and digital signatures and is the modern de facto standard.

Asymmetric encryption provides some significant strengths in comparison to its symmetric brethren. Asymmetric encryption can provide both confidentiality and nonrepudiation, and it solves the problems of key distribution and scalability. In fact, the only real downside to asymmetric—its weaknesses that you'll be asked about on the exam—is its performance (asymmetric is slower than symmetric, especially on bulk encryption) and processing power (because it usually requires a much longer key length, it's suitable for smaller amounts of data).

Hash Algorithms

Last in our discussion of algorithms are the hashing algorithms, which really don't encrypt anything at all. A

hashing algorithm is a *one-way* mathematical function that takes an input and typically produces a fixed-length string (usually a number), or hash, based on the arrangement of the data bits in the input. Its sole purpose in life is to provide a means to verify the integrity of a piece of data; change a single bit in the arrangement of the original data, and you'll get a different response.

What's in a Chain of Blocks?

If you're like me, naturally wary of virtually anything new, and extremely conservative with your finances and corresponding investment strategies, the advent of bitcoin sounded a bit like the old snake oil salesman at the carnival—just a little too good to be true, thank you very much. And certainly bitcoin has seen its fair share of controversy. It seems the currency of choice for many criminally minded folks, and if you're hit by ransomware, chances are you'll be asked for ransom in bitcoin. Not to mention the innumerable recent federal investigations into bitcoin variants as "pyramid schemes."

However, it wasn't all bad. In addition to the benefits of "mining" your own digital currency, and decentralizing currency overall (the concept of decentralizing authority and power of any kind appeals greatly to a whole bunch of us), the advent of the blockchain to track it all may turn out to be one of the greatest inventions/discoveries of this century. But, again, if you're like me, you simply didn't pay attention enough to blockchain as a whole, and are now sitting around wondering just what the heck it is that everyone is talking about? Well, fret not, dear reader, because the explanation—at least the basics, anyway—is easier to understand than you might've guessed.

Blockchain is defined as suite of distributed ledger technologies to track anything of value: in effect, it's a series of related transactions stored chronologically as blocks in a shared ledger. Each block of transactions in the ledger has a start and stop time (10 minutes in the bitcoin world), and stands as its own record. Therefore, if you wish to change the data in one block, you must first copy it, change the data, and then append it to the end of the chain—making traceback on changes to the data as a whole easy to discover. So what prevents someone from simply hacking in, grabbing a block, and changing it to whatever they want as the last record in the chain? Well, that's where blockchain gets really interesting...

The ledger for a given blockchain is "seen" by bunches of authorized computers, distributed all over the place. All systems in the network can see the ledger all at once, and all know the transactions that are flowing into and out of the current block being filled at any given time. When it's time to close the current block, all the systems in the network start working on a cryptographic puzzle—a giant, really difficult math problem. When a system—we'll call this computer Bender—solves the puzzle, it says, "Hey, it's me, Bender. You know, the loveable robot from *Futurama*? Yeah, I'm great. And I'm done, losers. I'm now going to close out this block with this list of transactions right here, and add it to the chain. Compare your computing cycles to mine and shut down, chumps."

Every other system in the network immediately becomes suspicious. They then stop everything and set about verifying two things. First, they verify Bender got the answer to the math problem correct, and, second, they verify that the list of transactions Bender wants to put in that block matches the list of transactions they themselves know about. As soon as more than half the systems in the network agree both of those are correct, Bender is allowed to close the block and add it to the chain. The next block then "opens," new transactions begin, and the whole scenario kicks off anew.

The benefits to this are, or at least should be, self-explanatory. Transparency with both the transactions themselves and with the ledger mean the end consumer can actually trust the data without having to go through a centralized authority. The implications for this use in things like medical

records, land titles, and finances, for just a few examples, are enormous. As an aside, if you'd like to see some of this in action, in real time, there are several blockchain viewing sites out there. Blockchain.info has tons of stuff available, and there are countless YouTube videos and articles for your perusal as well.

Will blockchain show up on your exam? Will blockchain take over the world? Will *Futurama* make yet another comeback on national television? Who really knows. Maybe I'll ask Professor Farnsworth to pull out the What-If machine and take a gander. I'll let you know what we see...

EXAM TIP The "one-way" portion of the hash definition is important. Although a hash does a great job of providing for integrity checks, it's not designed to be an encryption method. There isn't a way for a hash to be reverse-engineered.

For example's sake, suppose you have a small application you've developed and you're getting ready to send it off. You're concerned that it may get corrupted during transport and want to ensure the contents arrive exactly as you've created them. To protect it, you run the contents of the app through a hash, producing an output that reads something like this: EF1278AC6655BBDA93425FFBD28A6EA3. After e-mailing the link to download your app, you provide the hash for verification. Anyone who downloads the app can run it through the same hash program, and if the two values match, the app was downloaded successfully. If even a single bit was corrupted during transfer, the hash value would be wildly different.

Here are some examples of hash algorithms:

- **MD5 (Message Digest algorithm)** This produces a 128-bit hash value output, expressed as a 32-digit hexadecimal number. Created by Ronald Rivest, MD5 was originally popular for ensuring file integrity. However, serious flaws in the algorithm and the advancement of other hashes have resulted in this hash being rendered obsolete (U.S. CERT, August 2010). Despite its past, MD5 is still used for file verification on downloads and, in many cases, to store passwords.

- **SHA-1** Developed by the NSA, SHA-1 produces a 160-bit value output and was required by law for use in U.S. government applications. In late 2005, however, serious flaws became apparent, and the U.S. government began recommending the replacement of SHA-1 with SHA-2 after the year 2010 (see FIPS PUB 180-1).
- **SHA-2** This hash algorithm actually holds four separate hash functions that produce outputs of 224, 256, 384, and 512 bits. Although it was designed as a replacement for SHA-1, SHA-2 is still not as widely used.
- **SHA-3** This hash algorithm uses something called "sponge construction," where data is "absorbed" into the sponge (by XOR-ing the initial bits of the state) and then "squeezed" out (output blocks are read and alternated with state transformations).

EXAM TIP Another fun hash to remember? RIPEMD-# (RACE Integrity Primitives Evaluation Message Digest, where the # indicates the bit length). For example, RIPEMD-160 computes a 160-bit hash, RIPEMD-256 does 256 bits, and so on. It works through 80 stages, executing five blocks 16 times each. And then it does it again, finishing with something called modulo 32 addition (a math function used in cryptography).

A note of caution here: hashing algorithms are not impervious to hacking attempts, as is evidenced by the fact that they become outdated (cracked) and need replacing. The attack or effort used against hashing algorithms is known as a *collision* or a *collision attack*. Basically, a collision occurs when two or more files create the same output, which is not supposed to happen. When a hacker can create a second file that produces the same hash value output as the original, he may be able to pass off the fake file as the original, causing goodness knows what kinds of problems. Collisions, no

matter which hash we're discussing, are always a possibility. By definition, there are only so many combinations the hash can create given an input (MD5, for example, will generate only 2^128 possible combinations). Therefore, given the computation speed of modern computing systems, it isn't infeasible to assume you could re-create one. Matter of fact, you can even download tools to do it for you (www.bishopfox.com/resources/tools/other-free-tools/md4md5-collision-code/).

EXAM TIP You should know about the DUHK attack. "Don't Use Hard-coded Keys" (https://duhkattack.com/) refers to a vulnerability that allows attackers access to keys in certain VPN (virtual private network) implementations. It affects devices using the ANSI X9.31 random number generator (RNG) in conjunction with a hard-coded seed key.

For instance, one of the more common uses for a hash algorithm involves passwords. The original password is hashed; then the hash value is sent to the server (or whatever resource will be doing the authentication), where it is stored. When the user logs in, the password is hashed with the same algorithm and key; if the two match, then the user is allowed access. Suppose a hacker were to gain a copy of this hashed password and begin applying a collision attack to the value; that is, she compares data inputs and the hash values they present until the hashes match. Once the match is found, access is granted, and the hacker now holds the user's credentials. Granted, this can be defined as a brute-force attack (and when we get to password attacks later, you'll see this), but it is included here to demonstrate the whole idea—given a hash value for an input, you can duplicate it over time using the same hash and applying it to different inputs.

Sure, this type of attack takes a *lot* of time, but it's not unheard of. As a matter of fact, many of your

predecessors in the hacking field have attempted to speed things up for you by creating *rainbow tables* for just such a use. Because hackers must lead boring lives and have loads of time on their hands, lots of unscrupulous people sat down and started running every word, phrase, and compilation of characters they could think of into a hash algorithm. The results were stored in the rainbow table for use later. Therefore, instead of having to use all those computational cycles to hash your password guesses on your machine, you can simply compare the hashed file to the rainbow table. See? Isn't that easy?

NOTE In modern systems, rainbow table use may be effectively dead (http://blog.ircmaxell.com/2011/08/rainbow-table-is-dead.html). True, there's still a lot of debate, and many swear by them, but brute forcing using GPU-based systems has its advantages.

To protect against collision attacks and the use of rainbow tables, you can also use something called a salt (no, not the sodium chloride on your table in the cute little dispenser). This salt is much more virtual. A *salt* is a collection of random bits that are used as a key in addition to the hashing algorithm. Because the bits, and length, are random, a good salt makes a collision attack difficult to pull off. Considering that every time a bit is added to the salt it adds a power of 2 to the complexity of the number of computation involved to derive the outcome, you can see why it's a necessity in protecting password files.

NOTE Ever wonder why it's called a *salt?* While it's a point of some debate among some nerds, it probably originated from the practice of salting wells and mines throughout U.S. history. During the colonial period, salt was a valuable resource, and boiling huge vats of salt water was the primary collection method. Pouring a little salt into a well could then potentially greatly increase the value of a well. "Salting" a dead mine with a few gold flakes had the same effect.

Big Brother Gets Bold

If you've ever used a U.S. government system for any length of time, you've undoubtedly seen the big warning banner right at login. You know, the one that tells you everything you do should be for government work only, that certain activities are not allowed, and (the big one for our discussion) that you should have absolutely no expectation of privacy (in other words, everything you do is monitored and tracked). I guess most of us would expect that when using a government or business system—it's their network and resources, after all, so of course they would want to protect them. But what if you're using your own computer, on your home network, for your own purposes? Does the government have a right to see everything you send and receive?

It seems the answer to that question depends a lot of what you do for a living. Most of us cry foul and scream about our right to privacy, which is a valid point. Some of us, though, charged with the safety and security of the public, point out that it's difficult to combat terrorism and foul play when the bad guys are allowed to keep secrets. And Big Brother (the all-powerful, ever-watching government George Orwell warned us all about in *1984*) not only thinks your expectation of privacy is silly, it is actively pursuing your encryption keys to ensure its eyes are always open.

Here's a fun acronym for you: GAK. No, it's not just the green slimy stuff from Nickelodeon; it actually means government access to keys. Also referred to as *key escrow,* it's similar to the idea of wiretapping (a law enforcement agency can get court approval to listen to your phone calls). The concept is simple: software companies provide their encryption keys (or at least enough of the key that the remainder can be cracked) to the government, and the government promises to play nicely with them and use them only when it *really* needs to (that is, when a court issues a warrant).

Remember Edward Snowden—the famous ex-CIA and NSA employee who provided thousands of classified documents to the press, exposing what he felt were horrific invasion of privacy issues and abuses by the U.S. government. In response, the U.S. government pressured the e-mail service provider Lavabit to provide encryption key copies used to secure web, instant message, and e-mail traffic as part of its investigation. They refused and chose to shut down as a company rather than comply with the order, but that was GAK in action for everyone to see.

I'll leave it to you, dear reader, to form your own opinions about how far government tentacles should be allowed to spread and where the line of personal privacy becomes a hindrance to public safety. For example, the Australian Parliament recently passed the Assistance and Access Act, unopposed and unamended (https://www.eff.org/deeplinks/2018/12/new-fight-online-privacy-and-security-australia-falls-what-happens-next). The act basically states the government has the right to compel tech companies and developers to reengineer anything protected by cryptography in order for the government to use it (for spying purposes). Therefore, if Company A builds in a backdoor to comply with Australia's laws, who's to say that same backdoor wouldn't be made accessible in a different country via a court order?

People far smarter than me have framed this debate on both sides and know worlds more about it than I could ever dream. But I'm a paranoid guy by nature, so I'll caution you to remember one thing: Big Brother is watching—and can probably see more than you think.

EXAM TIP When it comes to questions on the exam regarding hashes, remember two things. First, they're used for integrity (any deviation in the hash value, no matter how small, indicates the original file has been corrupted). Second, even though hashes are one-way functions, a sufficient collision attack may break older versions (such as MD5).

Lastly on hashes, there are a bajillion different tools out there you can use to create and view them (and yes, *bajillion* is a real word). A few of note include HashCalc (www.slavasoft.com), MD5 Calculator (www.bullzip.com), and HashMyFiles (www.nirsoft.com). You can even get tools on your mobile device (like Hash Droid, from play.google.com) for your hashing needs on the go. Because who doesn't want to calculate some hash values while Instagramming or Snapchatting?

Steganography

While not an encryption algorithm in and of itself, steganography is a great way to send messages back and forth without others even realizing it. *Steganography* is the practice of concealing a message inside another medium (such as another file or an image) in such a way that only the sender and recipient even know of its existence, let alone the manner in which to decipher it. Think about it: in every other method we've talked about so far, anyone monitoring the wire *knows* you're trying to communicate secretly—they can see the cipher text and know something is up. With steganography, you're simply sending a picture of the kids fishing. Anyone watching the wire sees a cute picture and a lot of smiles, never knowing they're looking at a message saying, for instance, "People who eavesdrop are losers."

Steganography can be as simple as hiding the message in the text of a written correspondence or as complex as changing bits within a huge media file to carry a message. For example, you could let the recipient

know that each letter starting a paragraph is relevant. Or you could simply write in code, using names of famous landmarks to indicate a message. In another example, and probably closer to what most people associate steganography with, if you had an image file, you could simply change the least meaningful bit in every byte to represent data—anyone looking at it would hardly notice the difference in the slight change of color or loss of sharpness.

EXAM TIP How can you tell if a file is a stego-file? For text, character positions are key (look for text patterns, unusual blank spaces, and language anomalies). Image files will be larger in size, and may show some weird color palette "faults." Audio and video files require some statistical analysis and specific tools.

In image steganography, there are three main techniques, the first of which was just mentioned: least significant bit insertion. Another method is masking and filtering, which is usually accomplished on grayscale images. Masking hides the data in much the same way as a watermark on a document; however, it's accomplished by modifying the luminescence of image parts. Lastly, algorithmic transformation allows steganographers to hide data in the mathematical functions used in image compression. In any case, the image appears normal, except its file size is much bigger. To a casual observation, it might be nearly impossible to tell the image is carrying a hidden message. In a video or sound file, it may even be less noticeable.

If hiding messages in a single image file works, surely hiding messages in a giant video file will as well. Tools like OmniHide Pro and Masker do a good job of sticking messages into the video stream smoothly and easily. Audio steganography is just as effective, taking advantage of frequencies the human ear can't pick up—not to mention hiding data in a variety of other methods,

like phase encoding and tone insertion. DeepSound and MP3Stego are both tools that can assist with this.

Before you get all excited, though, and go running out to put secret messages in your cell phone pics from last Friday night's party, you need to know that a variety of tools and methods are in place to look for, and prevent, steganographic file usage. Although there are legitimate uses for it—digital watermarks (used by some companies to identify their applications) come to mind—most antivirus programs and spyware tools actively look for steganography. There are more "steg" or "stego" tools available than we could possibly cover here in this book, and they can be downloaded from a variety of locations (just be careful!). A few examples include QuickStego (quickcrypto.com), gifshuffle and SNOW (darkside.com.au), Steganography Studio (stegstudio.sourceforge.net), and OpenStego (www.openstego.info).

PKI, THE DIGITAL CERTIFICATE, AND DIGITAL SIGNATURES

So, we've spent some time discussing encryption algorithms and techniques as well as covering the theory behind it all. But what about the practical implementation? Just how does it all come together?

Well, there are a couple of things to consider in an overall encryption scheme. First is the protection of the data itself—the encryption. This is done with the key set—one for encrypting, one for decrypting. This may be a little bit of review here, but it's critical to realize the importance of key generation in an asymmetric encryption scheme. As we've already covered, two keys are generated for each party within the encryption scheme, and the keys are generated *as a pair*. The first key, used for encrypting message, is known as the *public key*. The second key, used for decrypting messages, is

known as the *private key*. Public keys are shared; private keys are not.

No pun intended here, I promise, but the key to a successful encryption system is the infrastructure in place to create and manage the encryption keys. Imagine a system with loose controls over the creation and distribution of keys—it would be near anarchy! Users wouldn't know which key was which, older keys could be used to encrypt and decrypt messages even though the user was gone, and the storage of key copies would be a nightmare. In a classic (and the most common) asymmetric encryption scheme, a public key and a private key, at a minimum, have to be created, managed, distributed, stored, and, finally, revoked.

Second, keep in mind that there's more to it than just encrypting and decrypting messages—there's the whole problem of nonrepudiation to address. After all, if you're not sure which public key actually belongs to the user Bill, what's the point of having an encryption scheme in the first place? You may wind up using the wrong key and encrypting a message for Bill that the bad guy can read with impunity—and Bill can't even open! There are multiple providers of encryption frameworks to accomplish this task, and most follow a basic template known as *public key infrastructure (PKI)*.

The PKI System

A friend of mine once told me that the classic PKI infrastructure is an example of "beautifully complex simplicity." PKI is basically a structure designed to verify and authenticate the identity of individuals within the enterprise taking part in a data exchange. It consists of hardware, software, and policies that create, manage, store, distribute, and revoke keys and digital certificates (which we'll cover in a minute). A simplified picture of the whole thing in action is shown in Figure 11-2, but be forewarned: not all PKI systems are identical. Some things are common among all PKI systems (for example,

the initial request for keys and certs is done in person), but there's lots of room for differences.

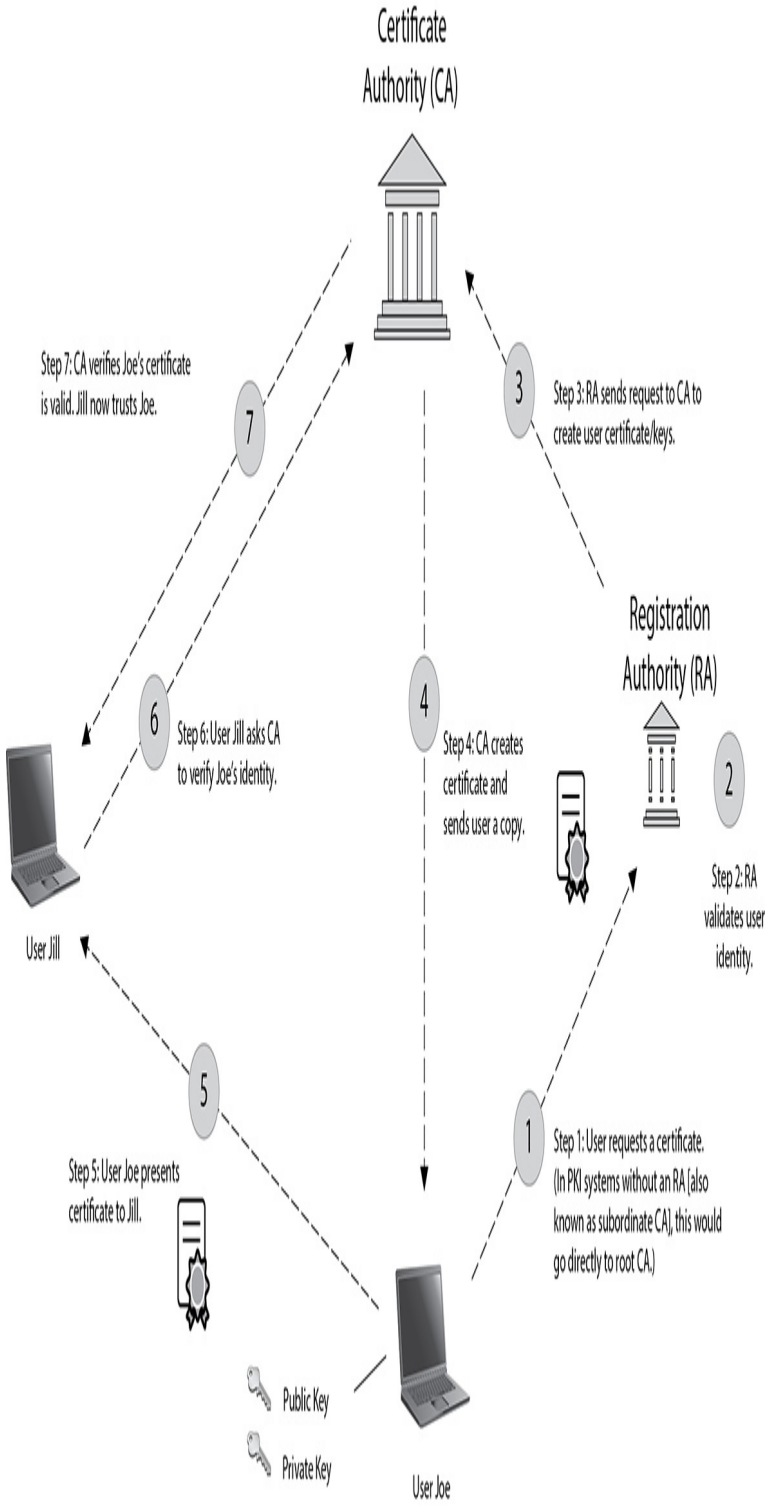


Figure 11-2 The PKI system

For one, the CA may be internal to begin with, and there could be any number of subordinate CAs—also known as *registration authorities (RAs)*—to handle things internally (as a matter of fact, most root CAs are removed from network access to protect the integrity of the system). In many systems, the public and private key pair—along with the certificate—are put on a token (like the Common Access Card [CAC] in the DoD), which is required going forward when the user wishes to authenticate. Additionally, certificates for applications and services are handled completely different. The whole thing can get confusing if you try to get it all at once. Just take it one step at a time and hopefully I'll answer everything along the way.

The system starts at the top, with a (usually) neutral party known as the *certificate authority (CA)*. The CA acts as a third party to the organization, much like a public notary; when it signs something as valid, you can trust, with relative assuredness, that it is. Its job is to create and issue digital certificates that can be used to verify identity. The CA also keeps track of all the certificates within the system (using a certificate management system) and maintains a *certificate revocation list (CRL)*, used to track which certificates have problems and which have been revoked.

NOTE In many PKI systems, an outside entity known as a *validation authority (VA)* is used to validate certificates—usually done via Online Certificate Status Protocol (OCSP).

The way the system works is fairly simple. Because the CA provides the certificate and key (public), the user can be certain the public key actually belongs to the intended recipient; after all, the CA is vouching for it. It also simplifies distribution of keys. A user doesn't have to go

to every user in the organization to get their individual keys; he can just go to the CA.

For a really simple example, consider user Jack, who just joined an organization without a full PKI system. Jack needs a key pair to encrypt and decrypt messages. He also needs a place to get the public keys for the other users on the network. With no controlling figure in place, he would simply create his own set of keys and distribute them in any way he saw fit. Other users on the network would have no real way of verifying his identity, other than, basically, to take his word for it. Additionally, Jack would have to go to each user in the enterprise to get their public key.

User Bob, on the other hand, joins an organization using a PKI structure with a local person acting as the CA. Bob goes to his security officer (the CA) and applies for encryption keys. The local security guy first verifies Bob is actually Bob (driver's license, and so on) and then asks how long Bob needs the encryption keys and for what purpose. Once he's satisfied, the CA creates the user ID in the PKI system, generating a key pair for encryption and a digital certificate for Bob to use. Bob can now send his certificate around, and others in the organization can trust it because the CA verifies it. Additionally, anyone wanting to send a message to Bob goes to the CA to get a legitimate copy of Bob's public key. It's much cleaner, much smoother, and much more secure. As an aside, and definitely worth pointing out here, the act of the CA creating the key is important, but the fact that the CA signs it digitally is what validates the entire system. Therefore, protection of your CA is of utmost importance.

NOTE Want more to worry about with the CA? Just imagine what could happen if an attacker manages to add a root CA for their own certificates into your browser. Once that's done, your browser will *automatically* trust certificates with that signature. It's not real common, but the browser tends to accept certificates signed by a trusted root. Root CAs are very important, and many people just assume that all the ones on their happy little Windows box are valid.

And finally, another term associated with PKI, especially when the topic is CAs, is *trust model*. This describes how entities within an enterprise deal with keys, signatures, and certificates, and there are three basic models. In the first, called *web of trust,* multiple entities sign certificates for one another. In other words, users within this system trust each other based on certificates they receive from other users on the same system.

EXAM TIP A certificate authority can be set up to trust a CA in a completely different PKI through something called *cross-certification.* This allows both PKI CAs to validate certificates generated from either side.

The other two systems rely on a more structured setup. A *single-authority system* has a CA at the top that creates and issues certificates. Users trust each other based on the CA. The *hierarchical trust system* also has a CA at the top (which is known as the *root* CA) but makes use of one or more registration authorities (subordinate CAs) underneath it to issue and manage certificates. This system is the most secure because users can track the certificate back to the root to ensure authenticity without a single point of failure.

Digital Certificates

I know this may seem out of order, since I've mentioned the word *certificate* multiple times already, but it's nearly impossible to discuss PKI without mentioning certificates, and vice versa. As you can probably tell so

far, a digital certificate isn't really involved with encryption at all. It is, instead, a measure by which entities on a network can provide identification. A digital certificate is an electronic file that is used to verify a user's identity, providing nonrepudiation throughout the system.

The certificate itself, in the PKI framework, follows a standard used worldwide. The X.509 standard, part of a much bigger series of standards set up for directory services and such, defines what should and should not be in a digital certificate. Because of the standard, any system complying with X.509 can exchange and use digital certificates to establish authenticity.

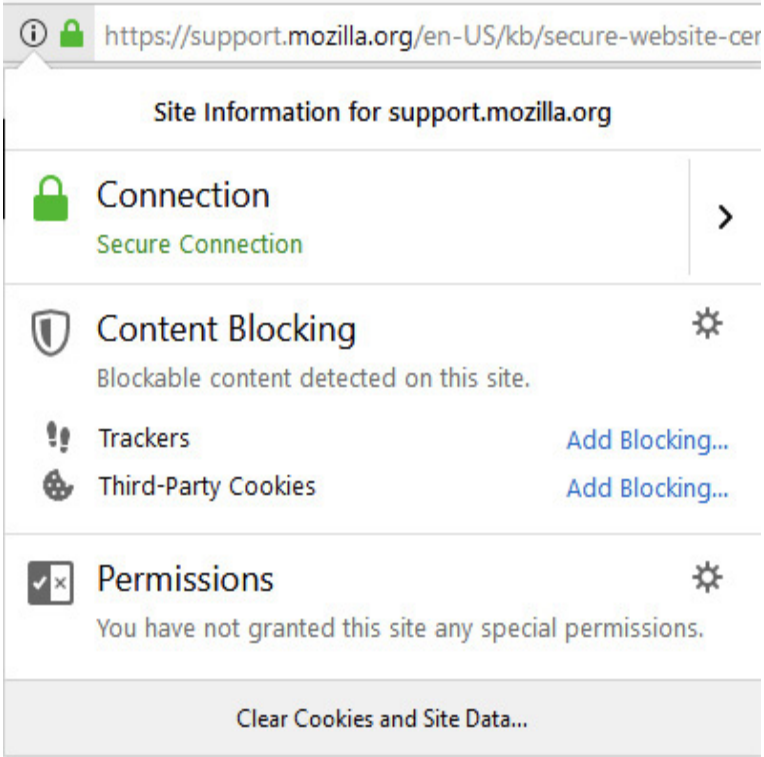
The contents of a digital certificate are listed here:

- **Version** This identifies the certificate format. Over time, the actual format of the certificate has changed slightly, allowing for different entries. The most common version in use is 1.
- **Serial Number** Fairly self-explanatory, the serial number is used to uniquely identify the certificate.
- **Subject** This is whoever or whatever is being identified by the certificate.
- **Algorithm ID (or Signature Algorithm)** This shows the algorithm that was used to create the digital signature.
- **Issuer** This shows the entity that verifies the authenticity of the certificate. The issuer is the one who creates the certificates.
- **Valid From and Valid To** These fields show the dates the certificate is good through.
- **Key Usage** This shows for what purpose the certificate was created.
- **Subject's Public Key** A copy of the subject's public key is included in the digital certificate, for obvious purposes.

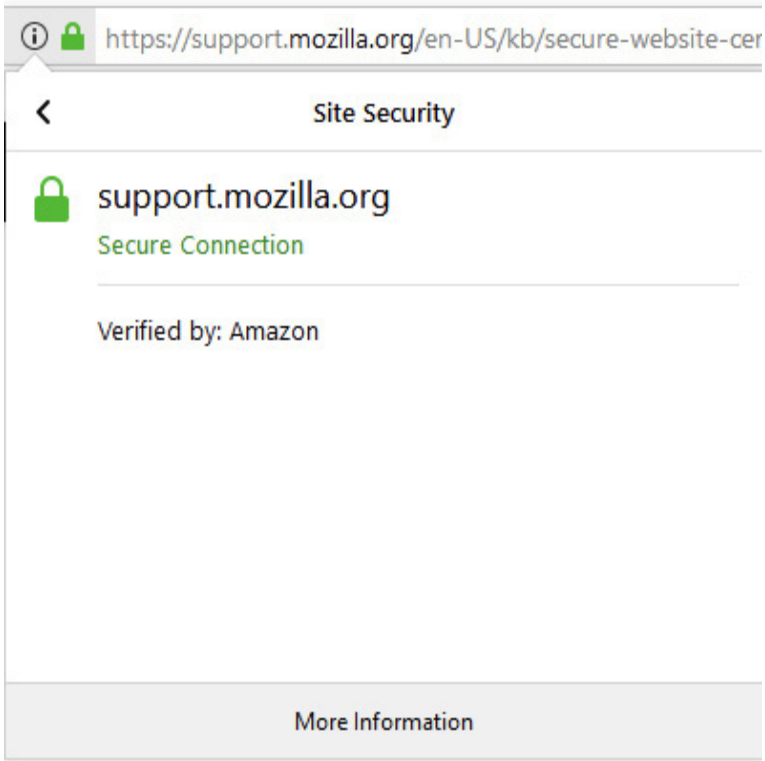
- **Optional fields** These fields include Issuer Unique Identifier, Subject Alternative Name, and Extensions.

To see them in action, try the steps listed here to look at a digital certificate (this one's actually from Mozilla). Any site using digital certificates will work; this one is simply used as an example:

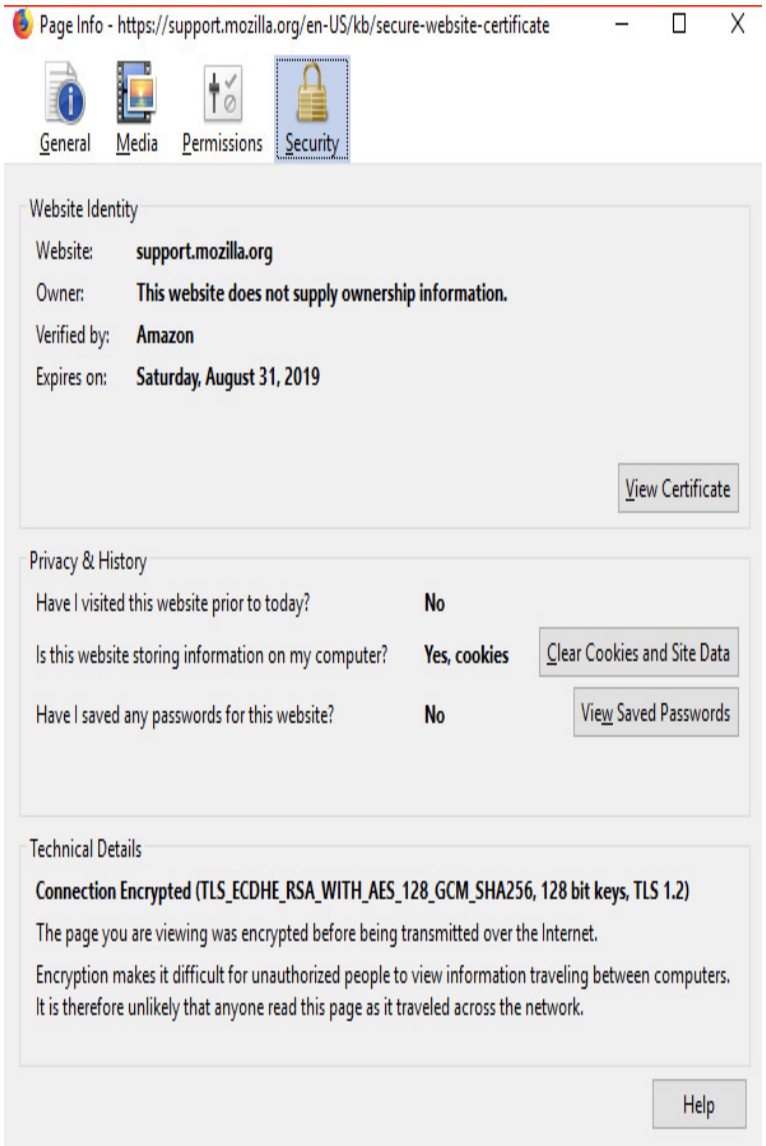
1. Open Firefox and go to https://support.mozilla.org/en-US/kb/secure-website-certificate (the site displayed gives a great rundown on digital certificates). Click the lock icon in the top-left corner and then click the right arrow beside Connection.



2. At the bottom of the next screen, click the More Information button.



3. When the page information appears, as shown in the following illustration, click View Certificate.



4. The digital certificate's General tab displays the certificate, as shown in the following illustration. The Details tab can show even more information.

Certificate Viewer: "support.mozilla.org"

General Details

This certificate has been verified for the following uses:

SSL Client Certificate

SSL Server Certificate

Issued To

Common Name (CN)	support.mozilla.org
Organization (O)	<Not Part Of Certificate>
Organizational Unit (OU)	<Not Part Of Certificate>
Serial Number	04:6A:CF:FE:FC:85:94:17:5C:E6:99:59:C6:72:9B:8B

Issued By

Common Name (CN)	Amazon
Organization (O)	Amazon
Organizational Unit (OU)	Server CA 1B

Period of Validity

Begins On	Monday, July 30, 2018
Expires On	Saturday, August 31, 2019

Fingerprints

SHA-256 Fingerprint	97:43:87:97:73:37:F0:35:50:38:BC:C7:0F:80:B2:78:90:1C:EC:40:D0:14:F1:C8:0E:B8:78:1C:83:C4:C6:6E
SHA1 Fingerprint	DF:1F:2A:33:07:17:95:7E:9C:37:46:E9:94:E7:19:29:3F:6C:4D:45

Close

EXAM TIP Know what is in the digital certificate and what each field does. It's especially important to remember the public key is sent with the certificate.

So, how does the digital certificate work within the system? For example's sake, let's go back to user Bob. He

applied for his digital certificate through the CA and anxiously awaits an answer. The cert arrives, and Bob notices two things: First, the certificate itself is signed. Second, the CA provided a copy of its own public key. He asks his security person what this all means.

Bob learns this method is used to deliver the certificate to the individual safely and securely and also provides a means for Bob to be *absolutely certain* the certificate came from the CA and not from some outside bad guy. How so? The certificate was signed by the CA before he sent it using the CA's *private* key. Because the only key in existence that could possibly decrypt it is the CA's own public key, which is readily available to anyone, Bob can rest assured he has a valid certificate. Bob can now use his certificate, containing information about him that others can verify with the CA, to prove his identity.

NOTE Speaking of root CAs, Microsoft Windows (and other operating systems) have certain companies and organizations they think are trustworthy, and they add these root CAs automagically for you. Which could be a rather terrifying thought if you were a paranoid guy like me. I mean, if a bad guy somehow dupes and/or compromises one of those root companies... It has happened before (https://www.csoonline.com/article/2623707/hacking/the-real-security-issue-behind-the-comodo-hack.html) and could cause the whole thing to collapse.

Finally, when it comes to certificates, you should also know the difference between signed certs and self-signed certs. Generally speaking, every certificate is signed by something, but the difference between these two comes down to who signed it and who validates it. As we've covered already, certificates can be used for tons of things, and each one is generated for a specific purpose. Suppose you have an application or service *completely* internal to your organization, and you want to provide

authentication services via certificates. A self-signed certificate—one created internally and never intended to be used in any other situation or circumstance—would likely be your best choice. In most enterprise-level networks, you're bound to find self-signed certificates all over the place. They save money and complexity—since there's no need to involve an external verification authority—and are relatively easy to put into place. Managing self-signed certs can sometimes be hard, and any external access to them is a definite no-no, but internal use is generally nodded at.

NOTE In the interest of covering everything, note that EC-Council seems to center on a self-signed certificate being signed by the same entity whose identity it certifies (that is, signed using the entity's own private key). In practice, internal CAs can be (and are) created to handle self-signed certs inside the network.

Signed certificates generally indicate a CA is involved and the signature validating the identity of the entity is confirmed via an external source—in some instances, a validation authority (VA). Signed certificates, as opposed to self-signed certificates, can be trusted: assuming the CA chain is validated and not corrupted, it's good everywhere. Obviously, anything accessible to (or using) external connectivity will require a signed certificate.

Digital Signatures

Speaking of signed and self-signed, let's take a few minutes to discuss the definition and description of the digital signature. The only real reason this is ever a confusing topic is because instructors spend a lot of time drilling into students' heads that the public key is for encryption and that the private key is for decryption. In general, this is a true statement (and I'm willing to bet you'll see it on your exam that way). However, remember

that the keys are created in pairs—what one key does, the other undoes. If you encrypt something with the public key, the private key is the only one that can decrypt it. But that works in reverse, too; if you encrypt something with your private key, your public key is the only thing that can decrypt it.

Keeping this in mind, the digital signature is an easy thing to understand. A digital signature is nothing more than an algorithmic output that is designed to ensure the authenticity (and integrity) of the sender—basically a hash algorithm. The way it works is simple:

1. Bob creates a text message to send to Joe.
2. Bob runs his message through a hash and generates an outcome.
3. Bob then encrypts the outcome of that hash with his *private* key and sends the message, along with the encrypted hash, to Joe.
4. Joe receives the message and attempts to decrypt the hash with Bob's *public* key. If it works, he knows the message came from Bob because the only thing Bob's public key could ever decrypt is something that was encrypted using his private key in the first place. Since Bob is the only one with that private key—voilà!

NOTE FIPS 186-2 specifies that something called the Digital Signature Algorithm (DSA) be used in the generation and verification of digital signatures. DSA is a Federal Information Processing Standard that was proposed by the National Institute of Standards and Technology (NIST) in August 1991 for use in its Digital Signature Standard (DSS).

When it comes to PKI, asymmetric encryption, digital certificates, and digital signatures, remembering a few important facts will solve a lot of headaches for you. Keys

are generated in pairs, and what one does, the other undoes. In general, the public key (shared with everyone) is used for encryption, and the private key (kept only by the owner) is used for decryption. Although the private key is created to decrypt messages sent to the owner, it is also used to prove authenticity through the digital signature (encrypting with the private key allows recipients to decrypt with the readily available public key). Key generation, distribution, and revocation are best handled within a framework, often referred to as PKI (public key infrastructure). PKI also allows for the creation and dissemination of digital certificates, which are used to prove the identity of an entity on the network and follow a standard (X.509).

ENCRYPTED COMMUNICATION AND CRYPTOGRAPHY ATTACKS

Okay, cryptography warriors, we're almost to the finish line. Hang with me—we have just a couple more things to get out of the way. They're important, and you will be tested on them, so don't ditch it all just yet. Thus far you've learned a little bit about what cryptography is and what encryption algorithms can do for you. In this section, we cover a few final pieces of the CEH cryptography exam objective: how people communicate securely with one another using various encryption techniques, and what attacks allow the ethical hacker to disrupt or steal that communication. But before we get there, let's take just a second to cover something really important—data at rest.

Data at rest (DAR) is a term being bandied about quite a bit lately in the IT security world, and it's probably one of the most misunderstood terms by senior management types. I say it's misunderstood because data "at rest" means different things to different people. In general terms, "at rest" means the data is not being accessed, and to many people that means everything on

the drive not currently being modified or loaded into memory. For example, a folder stored out on a server that's just sitting there would be at rest because "nobody is using it." But in reality there's more to the definition. The *true* meaning of data at rest is data that is in a stored state and not currently accessible. For example, data on a laptop when the laptop is powered off is in a resting state, and data on a backup drive sitting off the system/network is at rest, but data in a powered-on, networked, accessible server's folder is not—whether it's currently being used or not right now is immaterial.

DAR vendors are tasked with a simple objective: protect the data on mobile devices from loss or theft while it is in a resting state. Usually this entails full disk encryption (FDE), where pre-boot authentication (usually an account and password) is necessary to "unlock" the drive before the system can even boot up—once it's up and running, protection of the data falls to other measures. The idea is if a bad guy steals your laptop or mobile device, the data on the drive is protected. FDE can be software or hardware based, and it can use network-based authentication (Active Directory, for example) and/or local authentication sources (a local account or locally cached from a network source). Software-based FDE can even provide central management, making key management and recovery actions much easier. More than a few products and applications are available for doing this. Microsoft provides BitLocker on all operating system releases for exactly this purpose. McAfee has a full disk encryption offering called Endpoint Encryption, with administrative dashboards and controls. Symantec Drive Encryption and Gilisoft Full Disk Encryption are other options.

NOTE Another benefit to FDE is protection against the old boot-n-root attack. A bootable USB you can plug in to, boot off of, and then wreak havoc on the desktop system? Pfft—not only is the data protected, but the OS is, too.

Am I saying that files and folders on active systems don't require encryption protection? No, not at all—I'm simply pointing out that DAR protection is designed for a very specific purpose. Laptops and mobile devices should have full disk encryption because they are taken offsite and have the potential to be stolen. An HP Proliant DL80 on your data floor? Probably not, unless one of your admins takes it out of the cabinet, unhooks everything, and carries it home in the evening. And if they're doing that, you have some serious physical security issues to deal with.

Let's Go to the Source

Most of the time acronyms are just annoying to me. If I don't know what the letters in the acronym mean, I'll Google it and then add it to my repertoire of nerd lingo. Some, though, I not only know but hate viscerally, and DAR is one of them. Every time I see it my blood pressure rises, I start a facial tic I wasn't even aware I had, and I lose my inner monologue (a pop culture tip to Austin Powers fans).

I was talking about this section of the book with my lovely and talented wife on our walk today and was expressing my rage at not being able to convince upper management types (at a previous position) of its true definition when she said, "No, Matt, that's not right. SAN storage is data at rest." Tic, tic, tic, tic…

After our walk, we came back and, as we often do when we both think we're right, went to the source—in this case NIST. Two main sources were viewed: NIST SP 800-111 (http://csrc.nist.gov/publications/nistpubs/800-111/SP800-111.pdf) and NIST SP 800-53 (http://csrc.nist.gov/publications/nistpubs/800-53-Rev3/sp800-53-rev3-final_updated-errata_05-01-2010.pdf). As an aside, I had to go find a link for 800-53v4 myself because my wife was viewing a local copy on our home computer. When I asked her for the link and she said it was a local copy, I enquired why she had one stored locally, just sitting there. She responded, "Everyone should have a copy of NIST SP 800-53 on hand, why don't you?"

I love that woman.

In any case, what we found out is…we're both right. NIST SP 800-53 Control SC-28 doesn't actually define SAN or any other accessible network location as data at rest in the control itself, but does define desktops, laptops, mobile devices, and storage devices as data-at-rest locales—making me right. However, the control *enhancement*, SC-28 (1), does allow for system owners to include SAN and other locales in their data-at-rest control set. It's not required in the actual control for "high" security systems, but sometimes the enhancements are written to allow sys-tem owners some flexibility. In other words, organizations can define what is DAR and what isn't, to determine where they're at risk and to apply security controls appropriately—all of which made her right.

Interestingly, NIST doesn't even say you *must* encrypt these controls—it just says they must provide for confidentiality. Generally that involves some form of encryp-tion, but I'm sure somebody somewhere could argue some physical security controls and others could be used as data-at-rest protection. What's really important here is the level of flexibility involved in all of this. Just

keep in mind when you're discuss-ing this kind of stuff, there's often more than one right answer—especially if you're debating with my wife.

For the data on those servers that require additional confidentiality protection, of course encrypt the files or folder, or even the drives themselves, with a tool designed to help you with that specific security need. NIST gets into a lot of virtual disk and volume encryption, but I'm not sure that's all that valuable here. Instead, you should understand the difference between encrypting an entire disk with a pre-boot authenticating system (which changes the MBR) and individual volume, folder, and file encryption. For one tool example, Microsoft builds Encrypting File Systems (EFS) into its operating systems now for files, folders, and drives needing encryption. Others range from free products (such as VeraCrypt, AxCrypt, and GNU Privacy Guard) to using PKI within the system (such as Entrust products). The point is, full disk encryption may sound like a great idea in the boardroom, but once the drive is unlocked, the data inside is not protected.

Encrypted Communication

It's one thing to protect your data at rest, but it's another thing altogether to figure out how to transport it securely and safely. Encryption algorithms—both symmetric and asymmetric—were designed to help us do both, mainly because when all this (networking and the Internet) was being built, no one even thought security would be an issue.

Want proof? Name some application layer protocols in your head and think about how they work. SMTP? Great protocol, used to move e-mail back and forth. Secure? Heck no—it's all in plain text. What about Telnet and SNMP? Same thing, and maybe even worse (SNMP can do bad, bad things in the wrong hands). FTP? Please, don't even begin to tell me that's secure.

So, how can we communicate securely with one another? There are plenty of options, and I'm sure we could spend an entire book talking about them—but

we're not. The list provided here obviously isn't all-inclusive, but it does cover the major communications avenues and the major topics about them you'll need a familiarity with for your exam:

- **Secure Shell (SSH)** SSH is, basically, a secured version of Telnet. SSH uses TCP port 22, by default, and relies on public key cryptography for its encryption. Originally designed for remote sessions into Unix machines for command execution, it can be used as a tunneling protocol. SSH2 is the successor to SSH. It's more secure, efficient, and portable, and it includes a built-in encrypted version of FTP (SFTP).
- **Secure Sockets Layer (SSL)** This encrypts data at the transport layer, and above, for secure communication across the Internet. It uses RSA encryption and digital certificates and can be used with a wide variety of upper-layer protocols. SSL uses a six-step process for securing a channel, as shown in Figure 11-3. It is being largely replaced by Transport Layer Security (TLS).

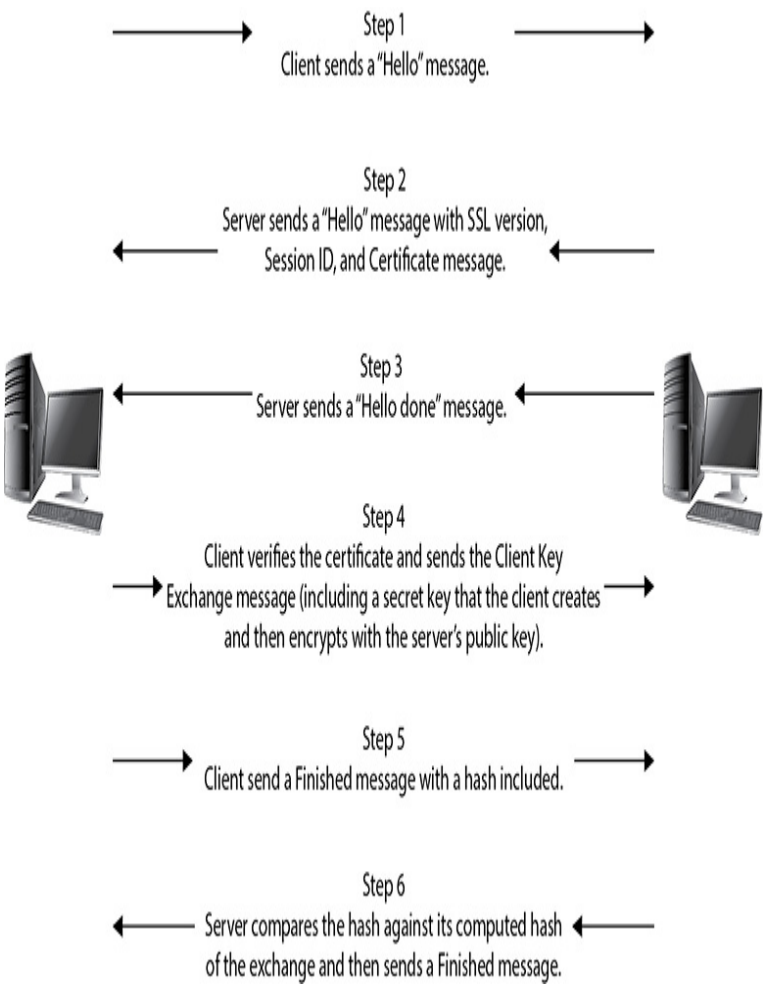


Figure 11-3 SSL connection steps

- **Transport Layer Security (TLS)** Using an RSA algorithm of 1024 and 2048 bits, TLS is the successor to SSL. The handshake portion (TLS Handshake Protocol) allows both the client and the server to authenticate to each other, and TLS Record Protocol provides the secured communication channel.
- **Internet Protocol Security (IPSec)** This is a network layer tunneling protocol that can be used in two modes: tunnel (entire IP packet encrypted) and transport (data payload encrypted). IPSec is capable of carrying nearly any application. The Authentication Header (AH) protocol verifies an IP packet's integrity and determines the validity of

its source: it provides authentication and integrity, but not confidentiality. Encapsulating Security Payload (ESP) encrypts each packet (in transport mode, the data is encrypted but the headers are not encrypted; in tunnel mode, the entire packet, including the headers, is encrypted).

- **PGP** Pretty Good Privacy was created way back in 1991 and is used for signing, compression, and encrypting and decrypting e-mails, files, directories, and even whole disk partitions, mainly in an effort to increase the security of e-mail communications. PGP follows the OpenPGP standard (RFC 4880) for encrypting and decrypting data. PGP is known as a hybrid cryptosystem, because it uses features of conventional and public key cryptography.

NOTE When e-mail is the topic, I'd be remiss in not mentioning S/MIME (Secure/Multipurpose Internet Mail Extensions). It was originally developed by RSA Data Security, Inc., and is a standard for public key encryption and signing of MIME data. The primary difference between PGP and S/MIME is that PGP can be used to encrypt not only e-mail messages but also files and entire drives.

Even though these are thought of as "secure" methods of communication, don't get too comfortable in using them—there's always room to worry. For example, it seems 2014 was a very bad year for SSL communications, as two very nasty exploits, Heartbleed and POODLE, apparently came out of nowhere. They caused veritable heart attacks and seemingly endless activity among security practitioners; not so coincidentally, they will show up on your exam multiple times. Let's take a look at each.

In late March of 2014, Google's security team was completing some testing of OpenSSL and discovered

something really terrible. Once they confirmed what they thought they'd found, Google notified OpenSSL on April 1, 2014 and, six days later, the public was notified of what *Forbes* cybersecurity columnist Joseph Steinberg said in his article "Massive Internet Security Vulnerability—Here's What You Need To Do" was "the worst vulnerability found (at least in terms of its potential impact) since commercial traffic began to flow on the Internet."

Heartbleed exploits a small feature in OpenSSL that turned out to present a very big problem. OpenSSL uses a heartbeat during an open session to verify that data was received correctly, and it does this by "echoing" data back to the other system. Basically, one system tells the other "I received what you sent and it's all good. Go ahead and send more." In Heartbleed, an attacker sends a single byte of data while telling the server it sent 64Kb of data. The server will then send back 64Kb of data—64Kb of random data from its memory.

EXAM TIP You can use the nmap command **nmap -d --script ssl-heartbleed --script-*args vulns.showall -sV [host]*** to search for the vulnerability: the return will say "State: NOT VULNERABLE" if you're good to go.

And what might be in this memory? The sky's the limit—user names and passwords, private keys (which are exceptionally troubling because future communication could be decrypted), cookies, and a host of other nifty bits of information are all in play. This would be concerning enough if the attack itself weren't so easy to pull off. Take a peak at the following code listing showing the use of the Metasploit auxiliary module openssl_heartbleed. Obviously, a few lines have been redacted to save some space, but it should be easy enough to see the module load, see some parameters set, initiate it by typing **exploit**, and return the 64Kb of

memory the server provides (the bolded text, for obvious reasons):

```
msf > use auxiliary/scanner/ssl/openssl_heartbleed
msf > auxiliary[openssl_heartbleed] > set RHOSTS 172.16.5.12
RHOSTS => 172.16.5.12
msf > auxiliary[openssl_heartbleed] > set RPORT 443
RPORT => 443
msf > auxiliary[openssl_heartbleed] > set THREADS 50
THREADS => 50
msf > auxiliary[openssl_heartbleed] > set verbose true
verbose => true
msf > auxiliary[openssl_heartbleed] > exploit
[*] 172.16.5.12:443 - Sending Client Hello...
[*] 172.16.5.12:443 - Sending Heartbeat
[*] 172.16.5.12:443 - Heartbeat response, 65551 bytes
[+] 172.16.5.12:443 - Heartbeat response with leak
[*] 172.16.5.12:443 - Printable info leaked:

S@$fy90Q6_fQH5f"!98532ED/AeL6.centos Firefox/3.6.24Accept: image/png,image/*;
q=0,8,*/*;q=0,5Accept Language: en-us,
en;q=0.5Accept-Encoding: gzip,deflateAccept-Charset:

ISO-8859-1,utf -8;q=0.7Kx
----Lines removed ----
----Lines removed ----
MA@1ba193il4bh366179b@k4user=matt&password=P@ssw0rd!$123&timezone

-offset=-5:NJ_,,BR ------
----Lines removed ----
```

Heartbleed caused major headaches and worry all over the world. Applications and organizations that were affected included multiple VMware products, Yahoo!, FileMaker, Cisco routers, HP server applications, SourceForge, and GitHub. And the problems weren't just on the commercial side: government agencies everywhere shut down online services while fix actions were put in place. And it's not over. Per AVG's Virus Labs, up to 1.5 percent of websites worldwide are still vulnerable, and there is no telling how many certificates

have not been updated/changed since the fix action (which may leave them vulnerable if private keys were stolen previously). Add to it “reverse Heartbleed” (where servers are able to perform the exact same thing in reverse, stealing data from clients) to compound the issue, and things are still very hairy.

EXAM TIP Another attack you may see referenced (now or in the near future) is FREAK. Factoring Attack on RSA-EXPORT Keys (FREAK) is a man-in-the-middle attack that forces a downgrade of an RSA key to a weaker length. The attacker forces the use of a weaker encryption key length, enabling successful brute-force attacks.

As if Heartbleed weren’t enough, POODLE (Padding Oracle On Downgraded Legacy Encryption) was (again) discovered by Google’s security team and announced to the public on October 14, 2014. This time it was a case of backward compatibility being a problem. The Transport Layer Security (TLS) protocol had largely replaced SSL for secure communication on the Internet, but many browsers would still revert to SSL 3.0 when a TLS connection was unavailable. They did this because many TLS clients performed a handshake effort, designed to degrade service until something acceptable was found. For example, the browser might offer TLS 1.2 first and, if it fails, retry and offer 1.0. Supposing a hacker could jump in the connection between client and server, he could interfere with these handshakes, making them all fail—which results in the client dropping to SSL 3.0.

NOTE Many of us who lean toward the conspiratorial side question the timing of these releases. Supposedly Google and Codenomicon discovered Heartbleed independently but both notified OpenSSL on the same date—April 1st. Six days later, the rest of us found out about it. Did companies

like Yahoo!, Google, and Microsoft have a chance to fix Heartbleed on their applications before the rest of the world got to hear about it? Makes you wonder, doesn't it? Especially since the paper announcing POODLE was released on October 14th, but the date on the release paper read *September*.

So what's the big deal? Well, it seems SSL 3.0 uses RC4, and that opens up a whole world of issues. SSL 3.0 has a design flaw that allows the padding data at the end of a block cipher to be changed so that the encryption cipher becomes less secure each time it is passed. Defined as "RC4 biases" in OpenSSL's paper on the subject (https://www.openssl.org/~bodo/ssl-poodle.pdf), if the same secret—let's say a password—is sent over several sessions, more and more information about it will leak. Eventually, the connection may as well be plain text (per the same source, an attacker need only make 256 SSL 3.0 requests to reveal 1 byte of encrypted messages), and the attacker sitting in the middle can see everything.

Mitigation for POODLE is straightforward: just don't use SSL 3.0 at all. Completely disabling SSL 3.0 on the client and server sides means the "degradation dance" can't ever take things down to SSL 3.0. Of course, in a recurring vein that frustrates and angers security professionals while simultaneously filling hackers with glee and joy, there are old clients and servers that just don't support TLS 1.0 and above. *[Insert sigh here.]* Therefore, you can implement TLS_FALLBACK_SCSV (a fake cipher suite advertised in the Client Hello message, which starts the SSL/TLS handshake) to hopefully prevent the attack.

NOTE Google's Chrome browser and Google servers already support TLS_FALLBACK_SCSV, with SSL 3.0 being removed completely. Fallback to SSL 3.0 was disabled in Chrome 39 (November 2014), and SSL 3.0 was disabled by default in Chrome 40 (January 2015). Mozilla disabled SSL

3.0 in Firefox 34 and ESR 31.3 (December 2014) and added TLS_FALLBACK_SCSV in Firefox 35.

Another mitigation is to implement something called "anti-POODLE record splitting." In short, this splits records into several parts, ensuring none of them can be attacked. However, although this may frustrate the exploit's ability to gather data, it also may cause compatibility issues due to problems in server-side implementations.

EXAM TIP Know Heartbleed and POODLE very, very well. Open SSL versions *1.0.1* and *1.0.1f* are vulnerable to Heartbleed, and its CVE notation is *CVE-2014-0160*. Be prepared for scenario-based questions involving SSL that will reference this attack—I guarantee you'll see them. POODLE (a.k.a. *PoodleBleed,* per EC-Council, CVE-2014-3566) will also appear in questions throughout your exam.

The last one we're going to visit before calling it a day is a doozy, and even though it hasn't made its way into the official courseware (and by extension your exam) as I write this, I guarantee it will soon. And I'd much rather give you more than you need now than to hear about me leaving something out later. As we've covered before, modern client/server communications use TLS, and SSL has been outdated. SSL 3.0, of course, had all sorts of problems and was disabled everywhere (other than in backward-compatibility-specific situations). But SSLv2? That's another story altogether.

It seems during all this hoopla, SSLv2 was… well…forgotten. Sure there were a few servers out there that still provided support for it, but for the most part that support didn't seem to matter to anyone. No up-to-date clients actually used SSLv2, so even though SSLv2 was known to be badly insecure, merely supporting it wasn't seen as a security problem. Right? If there's no client looking for it, then what difference does it make if it's there?

Pause for uproarious hacking laughter here, as we all contemplate something any first-year security student in Hardening of Systems 101 will state as an obvious step: turn off everything you're not using.

The DROWN (Decrypting RSA with Obsolete and Weakened eNcryption) attack, per the website DrownAttack.com, is a "serious vulnerability that affects HTTPS and other services that rely on SSL and TLS (essential cryptographic protocols for Internet security). DROWN allows attackers to break the encryption and read or steal sensitive communications, including passwords, credit card numbers, trade secrets, and financial data." When the CVE was released in 2016, up to 33 percent of Internet HTTPS servers tested were vulnerable to the attack.

Mitigation for DROWN is much like that for POODLE—turn off support for the offending encryption (in this case, SSLv2). Additionally, "server operators need to ensure that their private keys are not used anywhere with server software that allows SSLv2 connections. This includes web servers, SMTP servers, IMAP and POP servers, and any other software that supports SSL/TLS."

NOTE Remember way back in the beginning of this book I mentioned the balancing act between security and usability? There is no better example than the mitigations discussed here. Should you eliminate all backward compatibility in the name of security, you'll definitely ward off the occasional (and probably rare) attack, but you'll inevitably be faced with lots of "I can't get there because of security" complaints. Weigh your options carefully.

Cryptography Attacks

For the ethical hacker, all this information has been great to know and *is* important, but it's not enough just to know what types of encryption are available. What we need to know, what we're *really* interested in, is how to

crack that encryption so we can read the information being passed. A variety of methods and tools are available, and a list of the relevant ones are provided here for your amusement and memorization:

- **Known plain-text attack** In this attack, the hacker has both plain-text and corresponding cipher-text messages—the more, the better. The plain-text copies are scanned for repeatable sequences, which are then compared to the cipher-text versions. Over time, and with effort, this can be used to decipher the key.
- **Chosen plain-text attack** In a chosen plain-text attack, the attacker encrypts multiple plain-text copies himself in order to gain the key.
- **Adaptive chosen plain-text attack** The EC-Council definition for this is mind-numbingly obtuse: "the attacker makes a series of interactive queries, choosing subsequent plaintexts based on the information from the previous encryptions." What this really means is the attacker sends bunches of cipher texts to be decrypted and then uses the results of the decryptions to select different, closely related cipher texts. The idea is to gradually glean more and more information about the full target cipher text or about the key itself.
- **Cipher-text-only attack** In this attack, the hacker gains copies of several messages encrypted in the same way (with the same algorithm). Statistical analysis can then be used to reveal, eventually, repeating code, which can be used to decode messages later.
- **Replay attack** This is most often performed within the context of a man-in-the-middle attack. The hacker repeats a portion of a cryptographic exchange in hopes of fooling the system into setting up a communications channel. The attacker doesn't really have to know the actual data (such as the password) being exchanged; he

just has to get the timing right in copying and then replaying the bit stream. Session tokens can be used in the communications process to combat this attack.

- **Chosen cipher attack** In this attack, the bad guy (or good guy, depending on your viewpoint) chooses a particular cipher-text message and attempts to discern the key through comparative analysis with multiple keys and a plain-text version. RSA is particularly vulnerable to this attack.

EXAM TIP A side-channel attack isn't like the other traditional attacks mentioned. It is a physical attack that monitors environmental factors (like power consumption, timing, and delay) on the cryptosystem itself.

Along with these attacks, a couple of other terms are worth discussing here. *Man-in-the-middle* is another attack usually listed by many security professionals and study guides (depending on the test version you get, it may even be listed as such). Just keep in mind that this term simply means the attacker has positioned himself between the two communicating entities. Once there, he can launch a variety of attacks (interference, fake keys, replay, and so on). Additionally, the term *brute-force attack* is apropos to discuss in this context. Brute force refers to an attempt to try every possible combination against a target until successful. Although this can certainly be applied to cracking encryption schemes—and most commonly is defined that way—it doesn't belong *solely* in this realm (for example, it's entirely proper to say that using 500 people to test all the doors at once is a brute-force attack, as is sending an open request to every known port on a single machine).

NOTE An inference attack may not be what you think it is. *Inference* actually means you can derive information from the cipher text without actually decoding it. For example, if you are monitoring the encrypted line a shipping company uses and the traffic suddenly increases, you could assume the company is getting ready for a big delivery.

What's more, a variety of other encryption-type attack applications are waiting in the wings. Some applications, such as Carnivore and Magic Lantern (more of a keylogger than an actual attack application), were created by the U.S. government for law enforcement use in cracking codes. Some, such as L0phtcrack (used mainly on Microsoft Windows against SAM password files) and John the Ripper (a Unix/Linux tool for the same purpose), are aimed specifically at cracking password hashes. Others might be aimed at a specific type or form of encryption (for example, PGPcrack is designed to go after PGP-encrypted systems). A few more worth mentioning include CrypTool (www.cryptool.org), Cryptobench (www.addario.com), and Jipher (www.cipher.org.uk).

Regardless of the attack chosen or the application used to try it, it's important to remember that, even though the attack may be successful, attempts to crack encryption take a long time. The stronger the encryption method and the longer the key used in the algorithm, the longer the attack will take to be successful. Additionally, it's not an acceptable security practice to assign a key and never change it. No matter how long and complex the key, given a sufficient amount of time a brute-force attack will crack it. However, that amount of time can be from a couple of minutes for keys shorter than 40 bits to 50 or so years for keys longer than 64 bits. Obviously, then, if you combine a long key with a commitment to changing it within a reasonable time period, you can be

relatively sure the encryption is "uncrackable." Per the U.S. government, an algorithm using at least a 256-bit key cannot be cracked (see AES).

NOTE A truism of hacking really applies here: hackers are generally about the "low-hanging fruit." The mathematics involved in cracking encryption usually make it not worthwhile.

CHAPTER REVIEW

Cryptography is the science or study of protecting information, whether in transit or at rest, by using techniques to render the information unusable to anyone who does not possess the means to decrypt it. Plain-text data (something you can read) is turned into cipher-text data (something you can't read) by the application of some form of encryption. Encrypting data provides confidentiality because only those with the "key" can see it. Integrity can also be provided by hashing algorithms. Nonrepudiation is the means by which a recipient can ensure the identity of the sender and that neither party can deny having sent or received the message.

Encryption algorithms—mathematical formulas used to encrypt and decrypt data—are highly specialized and complex. There are two methods in which the algorithms actually work, and there are two methods by which keys can be used and shared. In stream ciphers, bits of data are encrypted as a continuous stream. In other words, readable bits in their regular pattern are fed into the cipher and are encrypted one at a time. These work at a high rate of speed. Block ciphers combine data bits into blocks and feed them into the cipher. Each block of data, usually 64 bits at a time, is then encrypted with the key and algorithm. These ciphers are considered simpler, and slower, than stream ciphers.

Symmetric encryption, also known as *single key* or *shared key,* simply means one key is used both to encrypt and to decrypt the data. It is considered fast and strong but poses some significant weaknesses. It's a great choice for bulk encryption because of its speed, but key distribution is an issue because the delivery of the key for the secured channel must be done offline. Additionally, scalability is a concern because as the network gets larger, the number of keys that must be generated goes up exponentially. DES, 3DES, Advanced Encryption Standard (AES), International Data Encryption Algorithm (IDEA), Twofish, and Rivest Cipher (RC) are examples.

Asymmetric encryption comes down to this: what the one key encrypts, the other key decrypts. It's important to remember the public key is the one used for encryption, whereas the private key is used for decryption. *Either can be used for encryption or decryption within the pair,* but in general remember public = encrypt, private = decrypt. Asymmetric encryption can provide both confidentiality and nonrepudiation and solves the problems of key distribution and scalability. The weaknesses include its performance (asymmetric is slower than symmetric, especially on bulk encryption) and processing power (asymmetric usually requires a much longer key length, so it's suitable for smaller amounts of data). Diffie-Hellman, Elliptic Curve Cryptosystem (ECC), El Gamal, and RSA are examples.

A hashing algorithm is a one-way mathematical function that takes an input and produces a single number (integer) based on the arrangement of the data bits in the input. It provides a means to verify the integrity of a piece of data—change a single bit in the arrangement of the original data, and you'll get a different response. The attack or effort used against a hashing algorithm is known as a collision or a collision attack. A collision occurs when two or more files create the same output, which is not supposed to happen. To

protect against collision attacks and the use of rainbow tables, you can also use a salt, which is a collection of random bits used as a key in addition to the hashing algorithm. MD5, SHA-1, SHA-2, and SHA-3 are examples of hash algorithms.

Steganography is the practice of concealing a message inside a text, image, audio, or video file in such a way that only the sender and recipient even know of its existence, let alone the manner in which to decipher it. Indications of steganography include character positions (in text files, look for text patterns, unusual blank spaces, and language anomalies) and large file sizes and color palette faults in image files. Audio and video files require some statistical analysis and specific tools.

In image steganography, there are three main techniques: least significant bit insertion, masking and filtering, and algorithmic transformation. Masking hides the data in much the same way as a watermark on a document; however, it's accomplished by modifying the luminescence of image parts. Algorithmic transformation allows steganographers to hide data in the mathematical functions used in image compression. Tools like OmniHide Pro and Masker do a good job of sticking messages into the video stream smoothly and easily. DeepSound and MP3Stego are both tools for audio steganography. Other tools include QuickStego (quickcrypto.com), gifshuffle and SNOW (darkside.com.au), Steganography Studio (stegstudio.sourceforge.net), and OpenStego (www.openstego.info).

PKI is a structure designed to verify and authenticate the identity of individuals within the enterprise taking part in a data exchange. It can consist of hardware, software, and policies that create, manage, store, distribute, and revoke keys and digital certificates. The system starts at the top, with a (usually) neutral party known as the certificate authority (CA) that creates and issues digital certificates. The CA also keeps track of all the certificates within the system and maintains a

certificate revocation list (CRL), used to track which certificates have problems and which have been revoked. The CA may be internal to begin with, and there could be any number of subordinate CAs—known as registration authorities (RAs)—to handle things internally (most root CAs are removed from network access to protect the integrity of the system). In many PKI systems, an outside entity known as a validation authority (VA) is used to validate certificates—usually done via Online Certificate Status Protocol (OCSP). A certificate authority can be set up to trust a CA in a completely different PKI through something called *cross-certification*. This allows both PKI CAs to validate certificates generated from either side.

CAs work in a *trust model*. This describes how entities within an enterprise deal with keys, signatures, and certificates, and there are three basic models. In the *web of trust,* multiple entities sign certificates for one another. In other words, users within this system trust each other based on certificates they receive from other users on the same system. A *single authority system* has a CA at the top that creates and issues certificates. Users trust each other based on the CA. The *hierarchical trust system* also has a CA at the top (which is known as the *root* CA) but makes use of one or more registration authorities (subordinate CAs) underneath it to issue and manage certificates. This system is the most secure because users can track the certificate back to the root to ensure authenticity without a single point of failure.

A digital certificate is an electronic file that is used to verify a user's identity, providing nonrepudiation throughout the system. The certificate typically follows the X.509 standard, which defines what should and should not be in a digital certificate. Version, Serial Number, Subject, Algorithm ID (or Signature Algorithm), Issuer, Valid From and Valid To, Key Usage, Subject's Public Key, and Optional are all fields within a digital certificate.

A self-signed certificate is one created and signed by the entity internally and never intended to be used in any other situation or circumstance. Signed certificates generally indicate a CA is involved and the signature validating the identity of the entity is confirmed via an external source—in some instances a validation authority (VA). Signed certificates, as opposed to self-signed certificates, can be trusted: assuming the CA chain is validated and not corrupted, it's good everywhere.

A digital signature is nothing more than an algorithmic output that is designed to ensure the authenticity (and integrity) of the sender. FIPS 186-2 specifies that the Digital Signature Algorithm (DSA) be used in the generation and verification of digital signatures. DSA is a Federal Information Processing Standard that was proposed by the National Institute of Standards and Technology (NIST) in August 1991 for use in their Digital Signature Standard (DSS). The steps in the use of a digital signature include the hashing of the message, with the result of the hash being encrypted by the sender's private key. The recipient then decrypts the hash result using the sender's public key, verifying the sender's identity.

Data at rest (DAR) is data that is in a stored state and not currently accessible. Protection of data on mobile devices from loss or theft while it is in a resting state usually entails full disk encryption (FDE), where pre-boot authentication (usually an account and password) is necessary to "unlock" the drive before the system can even boot up. FDE can be software or hardware based, and can use network-based authentication (Active Directory, for example) and/or local authentication sources (a local account or locally cached from a network source). Software-based FDE can even provide central management, making key management and recovery actions much easier.

Tools helpful in encrypting files and folders for other protective services include Microsoft Encrypted File Systems (EFS), VeraCrypt, AxCrypt, and GNU Privacy

Guard. The point is, full disk encryption may sound like a great idea in the boardroom, but once the drive is unlocked, the data inside is not protected.

Encrypted communication methods include the following:

- **Secure Shell (SSH)** A secured version of Telnet, using TCP port 22, by default, and relying on public key cryptography for its encryption.
- **Secure Sockets Layer (SSL)** Encrypts data at the transport layer and above, for secure communication across the Internet. It uses RSA encryption and digital certificates and can be used with a wide variety of upper-layer protocols. SSL uses a six-step process for securing a channel.
- **Transport Layer Security (TLS)** Uses an RSA algorithm of 1024 and 2048 bits; TLS is the successor to SSL.
- **Internet Protocol Security (IPSec)** Network layer tunneling protocol that can be used in two modes: tunnel (entire IP packet encrypted) and transport (data payload encrypted).
- **PGP (Pretty Good Privacy)** Used for signing, compression, and encrypting and decrypting e-mails, files, directories, and even whole disk partitions, mainly in an effort to increase the security of e-mail communications.

Heartbleed and POODLE were successful attacks against secure communications. Heartbleed exploits the heartbeat feature in OpenSSL, which tricks the server into sending 64Kb of data from its memory. You can use the nmap command **nmap -d --script ssl-heartbleed --script-*args vulns.showall -sV [host]*** to search for the vulnerability: the return will say "State: NOT VULNERABLE" if you're good to go. The Metasploit auxiliary module openssl_heartbleed can be

used to exploit this. Open SSL versions *1.0.1* and *1.0.1f* are vulnerable, and the CVE notation is *CVE-2014-0160*.

POODLE (Padding Oracle On Downgraded Legacy Encryption) took advantage of backward-compatibility features in TLS clients, allowing sessions to drop back to the vulnerable SSL 3.0, which has a design flaw that allows the padding data at the end of a block cipher to be changed so that the encryption cipher becomes less secure each time it is passed. Defined as "RC4 biases," if the same secret is sent over several sessions, more and more information about it will leak. Mitigation for POODLE is to not use SSL 3.0 at all. You can implement TLS_FALLBACK_SCSV (a fake cipher suite advertised in the Client Hello message, which starts the SSL/TLS handshake) on areas that must remain backward compatible. Another mitigation is to implement something called "anti-POODLE record splitting." In short, this splits records into several parts, ensuring none of them can be attacked. However, although this may frustrate the exploit's ability to gather data, it also may cause compatibility issues due to problems in server-side implementations.

Cipher attacks fall into a few categories and types. Known plain-text attacks, cipher-text-only attacks, and replay attacks are examples. Man-in-the-middle is usually listed as a type of attack by many security professionals and study guides (depending on the test version you get, it may even be listed as such). Just keep in mind that a man-in-the-middle situation simply means the attacker has positioned himself between the two communicating entities. *Brute force* refers to an attempt to try every possible combination against a target until successful.

QUESTIONS

1. Which of the following attacks acts as a man in the middle, exploiting fallback mechanisms in TLS clients?

   A. POODLE

B. Heartbleed

C. FREAK

D. DROWN

2. RC4 is a simple, fast encryption cipher. Which of the following is *not* true regarding RC4?

A. RC4 can be used for web encryption.

B. RC4 uses block encryption.

C. RC4 is a symmetric encryption cipher.

D. RC4 can be used for file encryption.

3. An organization has decided upon AES with a 256-bit key to secure data exchange. What is the primary consideration for this?

A. AES is slow.

B. The key size makes data exchange bulky and complex.

C. It uses a shared key for encryption.

D. AES is a weak cypher.

4. Joe and Bob are both ethical hackers and have gained access to a folder. Joe has several encrypted files from the folder, and Bob has found one of them unencrypted. Which of the following is the best attack vector for them to follow?

A. Cipher text only

B. Known plain text

C. Chosen cipher text

D. Replay

5. You are reviewing security plans and policies, and you wish to provide protection to organization laptops. Which effort listed protects system folders, files, and MBR until valid credentials are provided at pre-boot?

A. Cloud computing

B. SSL/TLS

C. Full disk encryption

D. AES

6. Which of the following is used to distribute a public key within the PKI system, verifying the user's identity to the recipient?

A. Digital signature

B. Hash value

C. Private key

D. Digital certificate

7. A hacker feeds plain-text files into a hash, eventually finding two or more that create the same fixed-value hash result. This anomaly is known as what?

A. Collision

B. Chosen plain text

C. Hash value compromise

D. Known plain text

8. An attacker uses a Metasploit auxiliary exploit to send a series of small messages to a server at regular intervals. The server responds with 64Kb of data from its memory. Which of the following best describes the attack being used?

A. POODLE

B. Heartbleed

C. FREAK

D. DROWN

9. Which of the following statements is true regarding encryption algorithms?

A. Symmetric algorithms are slower, are good for bulk encryption, and have no scalability problems.

B. Symmetric algorithms are faster, are good for bulk encryption, and have no scalability problems.

C. Symmetric algorithms are faster, are good for bulk encryption, but have scalability problems.

D. Symmetric algorithms are faster but have scalability problems and are not suited for bulk encryption.

10. Within a PKI system, Julia encrypts a message for Heidi and sends it. Heidi receives the message and decrypts the message using what?

A. Julia's public key

B. Julia's private key

C. Heidi's public key

D. Heidi's private key

11. Which of the following is a symmetric encryption method that transforms a fixed-length amount of plain text into an encrypted version of the same length?

A. Stream

B. Block

C. Bit

D. Hash

12. Which symmetric algorithm uses variable block sizes (from 32 to 128 bits)?

A. DES

B. 3DES

C. RC

D. MD5

13. Which hash algorithm produces a 160-bit output value?

A. SHA-1

B. SHA-2

C. Diffie-Hellmann

D. MD5

14. Two different organizations have their own public key infrastructure up and running. When the two companies merged, security personnel wanted both PKIs to validate certificates from each other. What

must the CAs for both companies establish to accomplish this?

A. Key exchange portal

B. Key revocation portal

C. Cross-site exchange

D. Cross-certification

15. Within a PKI, which of the following verifies the applicant?

A. Registration authority

B. User authority

C. Revocation authority

D. Primary authority

16. Which of the following is a software application used to asymmetrically encrypt and digitally sign e-mail?

A. PGP

B. SSL

C. PPTP

D. HTTPS

ANSWERS

1. **A.** In a POODLE attack, the man in the middle interrupts all handshake attempts by TLS clients, forcing a degradation to a vulnerable SSL version.
2. **B.** RC4 is a simple, fast, symmetric stream cipher. It can be used for almost everything you can imagine an encryption cipher could be used for (you can even find it in WEP).
3. **C.** AES is a symmetric algorithm, which means that the same key is used for encryption and decryption. The organization will have to find a secured means to transmit the key to both parties before any data exchange.
4. **B.** In a known plain-text attack, the hacker has both plain-text and cipher-text messages; the

plain-text copies are scanned for repeatable sequences, which are then compared to the cipher-text versions. Over time, and with effort, this can be used to decipher the key.

5. **C.** FDE is the appropriate control for data-at-rest protection. Pre-boot Authentication provides protection against loss or theft.
6. **D.** A digital certificate contains, among other things, the sender's public key, and it can be used to identify the sender.
7. **A.** When two or more plain-text entries create the same fixed-value hash result, a collision has occurred.
8. **B.** Heartbleed takes advantage of the data-echoing acknowledgement heartbeat in SSL. OpenSSL version 1.0.1 through version 1.0.1f are vulnerable to this attack.
9. **C.** Symmetric algorithms are fast, are good for bulk encryption, but have scalability problems.
10. **D.** Heidi's public key is used to encrypt the message. Her private key is used to decrypt it.
11. **B.** Block encryption takes a fixed-length block of plain text and converts it to an encrypted block of the same length.
12. **C.** Rivest Cipher (RC) uses variable block sizes (from 32 to 128 bits).
13. **A.** SHA-1 produces a 160-bit output value.
14. **D.** When PKIs need to talk to one another and trust certificates from either side, the CAs need to set up a mutual trust known as *cross-certification*.
15. **A.** A registration authority (RA) validates an applicant into the system, making sure they are real, valid, and allowed to use the system.
16. **A.** Pretty Good Privacy (PGP) is used for signing, compression, and encrypting and decrypting e-mails, files, directories, and even whole disk